SIMATS ENGINEERING
ASSESSMENT TOOL 2

COURSE NAME: Artificial Intelligence **COURSE CODE: CSA1701**

COURSE OUTCOME COVERED

CO2: *Experiment search and game-solving techniques such as Greedy Search, A*, CSP, Minimax, and Alpha-Beta pruning with heuristic functions for solving complex AI problems efficiently. (BL3)*

Assessment Tool Weightage: 50%

Duration: 1 Hour

QUESTIONS: 5 **Total Marks:50**

Annexure A

Scenario Based Assignment

Code of Conduct:

I JAYA PRAKASH M / 1912260496 certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.
I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student: JAYA PRAKASH M

Annexure A

Scenario Based Assignment

1. **Scenario** : A government deploys AI-powered drones to deliver emergency medicines in a flood-affected region. The environment is highly dynamic—roads may suddenly become inaccessible, and weather conditions affect travel cost. The drone has access to: A heuristic estimate (straight-line distance), Partial real-time updates about blocked paths and Variable movement costs depending on terrain and weather. However, due to urgency, the drone must quickly decide routes with minimum delay and risk.

Tasks:

i. Explain how Greedy Best-First Search would behave in this scenario and discuss its strengths and weaknesses under uncertainty

ii. Explain how A* would handle the same situation, including how it balances cost and heuristic iii. Analyze the impact of heuristic accuracy on both algorithms iv. Discuss how dynamic changes (blocked paths) affect search performance

v. Recommend the most suitable algorithm and justify your decision considering realtime constraints, optimality, and safety

2. **Scenario**: A smart city implements an AI system to optimize traffic signal timings across hundreds of intersections. The objective is to minimize Total waiting time, Fuel consumption and Traffic congestion. The system must continuously adjust signals based on traffic flow. However the search space is extremely large, traffic patterns change dynamically and the system may get stuck in locally optimal solutions

Tasks. Formulate this as an optimization problem and explain why traditional search is inefficient

i. Explain how Hill Climbing would work in this scenario and identify its limitations ii. Discuss issues like local maxima, plateaus, and ridges with real-world interpretation

iii. Explain how Simulated Annealing or Genetic Algorithms can overcome these limitations

iv. Recommend the best approach and justify your choice based on scalability and adaptability

3. **Scenario:** An autonomous rover is deployed on Mars to explore unknown terrain. It must: navigate safely to collect samples, avoid hazards (craters, rocks), operate with limited sensing capability and make decisions without a complete map. The rover updates its knowledge as it explores, making this a real-time decision-making problem.

Tasks:

i. Explain why this problem requires an online search agent instead of offline search ii. Analyze the challenges of partial observability and dynamic environments

iii. Describe how the rover builds and updates its internal model iv. Compare online search with uninformed and informed search strategies

v. Justify the most suitable approach considering uncertainty, adaptability, and safety

4. **Scenario:** A university is designing an AI system to automatically generate exam timetables. The system must satisfy multiple constraints: No student should have overlapping exams, Limited number of exam halls, Certain subjects must be scheduled before others, Faculty availability constraints, Special accommodations for some students, The complexity increases as the number of courses and students grows.

Tasks :

Formulate the problem as a Constraint Satisfaction Problem (CSP)

i. Clearly define variables, domains, and constraints with explanation ii. Explain how backtracking search works in this scenario iii. Discuss how constraint propagation (e.g., forward checking) improves efficiency iv. Analyze real-world challenges and justify the best strategy for scalability

5. **Scenario:** Strategic Game AI for Real-Time Decision Making (Minimax & Alpha-Beta Pruning) A gaming company is developing an AI opponent for a real-time strategy game. The AI must complete against human players, Make decisions within strict time limits, Evaluate complex game states with multiple possible moves, The decision tree is extremely large, making computation expensive.

Tasks:

i. Explain how the Minimax algorithm models this problem ii. Analyze the computational challenges of large game trees iii. Explain how Alpha-Beta pruning improves efficiency with detailed reasoning iv. Design an evaluation function and justify the factors considered

v. Recommend a strategy for balancing optimality and real-time performance

RUBRICS FOR CALCULATION

Criteria	Weightage (%)	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)
Understanding of Scenario	20%	Demonstrates clear and complete understanding of the scenario and context	Shows good understanding with minor gaps	Partial understanding; misses key aspects	Misinterprets or fails to understand the scenario
Identification of Key Issues	20%	Accurately identifies all relevant issues and factors involved	Identifies most key issues with minor omissions	Identifies limited issues; some irrelevant points	Unable to identify key issues
Application of Concepts	25%	Applies appropriate concepts effectively to address the scenario	Applies concepts with minor errors	Limited or partially correct application	Unable to apply relevant concepts
Decision Making	20%	Makes wellreasoned, logical decisions with strong rationale	Makes reasonable decisions with some justification	Makes weak decisions with limited justification	No clear decisions made or rationale provided
Clarity of Response	15%	Response is clear, structured, and logically presented	Generally clear with minor issues	Somewhat unclear or poorly structured	Disorganized and difficult to understand

CO2-AT-2

ANNEXURE A

SCENARIO-BASED ASSIGNMENT

1. AI-Powered Drones for Emergency Medicine Delivery

i. Greedy Best-First Search: Behaviour, Strengths and Weaknesses

Greedy Best-First Search selects the next path based only on the heuristic value $h(n)$, which estimates the distance from the current location to the destination.

- Behaviour: The drone selects the path that appears closest to the destination based on straight-line distance.
- Strengths: It is simple, fast, and can quickly identify promising routes.
- Weaknesses: It ignores the cost already incurred and may select a blocked or dangerous route.
- Under uncertainty: Inaccurate weather information or unexpected obstacles may cause delays and repeated route planning.

ii. A* Search: Cost and Heuristic

A* Search considers both the actual cost of travelling and the estimated remaining cost.

The evaluation function is:

$$ f(n)=g(n)+h(n) $$

Where:

- $g(n)$ = actual cost from the starting point to the current node.
- $h(n)$ = estimated cost from the current node to the destination.
- $f(n)$ = estimated total cost of the path.

The drone selects the route with the lowest estimated total cost. It can consider weather, terrain, fuel consumption, and travel time. If the heuristic is admissible and the relevant costs are correctly modelled, A* can find an optimal route in a static environment.

iii. Impact of Heuristic Accuracy

A good heuristic improves the performance of both algorithms.

- Greedy Best-First Search: An accurate heuristic helps the drone move towards the destination quickly, but it does not guarantee the lowest-cost route.
- A Search:* An accurate heuristic reduces unnecessary exploration and helps identify an optimal route when the required conditions are satisfied.
- Poor heuristic: Both algorithms may explore unsuitable routes or require additional computation.

iv. Impact of Dynamic Changes

Blocked roads, storms, and changing terrain costs can make the original route unsuitable. The drone must update its map and recalculate its route.

Greedy Best-First Search may quickly select a new route, but that route might be risky or expensive. A* can calculate a route that considers both the cost already incurred and the estimated remaining cost. Replanning methods can reduce the computation required after changes.

v. Recommended Algorithm

A Search with real-time replanning* is recommended because it considers both route cost and estimated distance. Safety constraints should be included in the route-planning system, and the drone should use updated weather and obstacle information.

For strict time limits, an incremental method such as Lifelong Planning A* can help reuse previous search results. If immediate safety is threatened, the drone should prioritize collision avoidance and a safe fallback route.

2. Smart City Traffic Signal Optimization

Formulation as an Optimization Problem

The objective is to minimize total traffic waiting time, fuel consumption, and congestion across hundreds of intersections.

A general objective function is:

$$\min J=w_1T+w_2F+w_3C$$

Where:

- T = total vehicle waiting time.
- F = fuel consumption.
- C = traffic congestion.
- w_1,w_2,w_3 = weights representing the importance of each objective.

The constraints include minimum and maximum green-light durations, pedestrian crossing times, traffic safety rules, and coordination between intersections.

Traditional exhaustive search is inefficient because the number of possible signal schedules grows rapidly with the number of intersections. Traffic conditions also change continuously, making repeated exhaustive calculations impractical.

i. Hill Climbing Algorithm

Hill Climbing starts with an initial signal schedule and repeatedly selects a neighbouring schedule that improves the objective function.

Example: If changing a traffic signal's green-light duration reduces waiting time, the system accepts that change. It continues until no neighbouring schedule provides an improvement.

Limitations:

- It may become trapped in a local optimum.
- It may fail to find a better solution that requires several changes together.
- It may need frequent restarting when traffic conditions change.

ii. Local Maxima, Plateaus and Ridges

- Local maxima or minima: A signal schedule performs better than nearby schedules but is not the best overall solution. For a minimization problem, the relevant term is a local minimum.
- Plateaus: Several neighbouring schedules have nearly identical performance, so the algorithm struggles to identify a better direction.

- Ridges: Improving traffic flow may require coordinated changes across multiple intersections. Changing only one signal at a time may not improve the result.

iii. Simulated Annealing and Genetic Algorithms

Simulated Annealing (SA): It sometimes accepts a worse solution to escape a local optimum. This acceptance is more likely at higher temperatures and becomes less likely as the temperature decreases.

Genetic Algorithm (GA): It maintains a population of possible signal schedules. Selection, crossover, and mutation produce new schedules, and the system evaluates them using the objective function.

Both methods can explore a larger search space than basic Hill Climbing.

iv. Recommended Approach

A Genetic Algorithm combined with real-time traffic monitoring and local optimization is suitable for a large smart city.

- It explores multiple possible schedules.
- It can optimize several objectives simultaneously.
- It supports coordinated changes across intersections.
- It can adapt to updated traffic measurements by periodically reevaluating candidate schedules.

For immediate traffic changes, local optimization can adjust individual signals while the Genetic Algorithm searches for improved network-wide schedules. Safety and pedestrian constraints must always be enforced.

3. Autonomous Mars Rover Using Online Search

i. Why Online Search Is Required

Online search is suitable because the rover explores an unknown environment and cannot obtain a complete map before starting its journey.

The rover discovers rocks, craters, slopes, and sample locations while moving. It must make decisions using the information currently available and revise its plans as new information arrives.

Offline search assumes that the relevant search space and action outcomes are sufficiently known in advance. This assumption does not hold for the rover's unknown terrain.

ii. Challenges of Partial Observability and Dynamic Environments

Partial observability:

- Sensors provide incomplete information about the terrain.
- Obstacles may be hidden behind rocks.
- Sensor measurements may contain errors.
- The rover may not know the exact location of a hazard.

Dynamic environment:

- Dust storms may reduce visibility.
- Terrain conditions may change.
- Communication delays may prevent immediate instructions from Earth.
- Limited energy and processing power restrict the number of calculations.

These challenges require continuous sensing, risk assessment, and replanning.

iii. Building and Updating the Internal Model

The rover maintains an internal representation of its environment using sensor readings and previous observations.

The process is:

1. Sense: Collect information from cameras, distance sensors, and other instruments.
2. Estimate: Determine its position and identify nearby obstacles.
3. Map: Record known safe paths, hazards, and unexplored areas.
4. Plan: Select a route towards the target or sample location.
5. Act: Move a short distance along the selected route.
6. Update: Compare new observations with the existing map and revise the plan.

This process allows the rover to improve its understanding of the environment over time.

iv. Comparison of Search Strategies

Search strategy	Description	Suitability
Uninformed search	Uses no heuristic information	May explore many unnecessary locations
Informed search	Uses a heuristic to guide exploration	Efficient when a useful heuristic is available
Online search	Alternates between acting, observing, and planning	Suitable for unknown environments

Online search can use informed or uninformed methods internally. For example, the rover can use heuristic-guided local planning while continuously updating its map.

v. Recommended Approach

Online, heuristic-guided search with continuous replanning and safety checks is recommended.

The rover should use sensor information to identify safe paths, prioritize reachable sample locations, avoid hazardous terrain, and conserve energy. It should also maintain safe fallback actions when sensor readings are uncertain.

This approach provides adaptability, supports decisions in unknown environments, and improves safety compared with relying on a fixed route.

4. University Exam Timetable Using a Constraint Satisfaction Problem (CSP)

Formulation as a CSP

A Constraint Satisfaction Problem consists of variables, domains, and constraints. The objective is to assign a valid time and examination hall to each course without violating the university's requirements.

i. Variables, Domains and Constraints

Variables:

For every course C_i, define:

- T_i: examination time slot.
- R_i: examination hall.

Domains:

- T_i can be any available examination slot.
- R_i can be any suitable examination hall with sufficient capacity and accessibility.

Constraints:

1. Student conflict: Two exams taken by the same student cannot occur at the same time.
2. Hall capacity: The selected hall must accommodate all students taking the exam.
3. Hall availability: A hall cannot host two exams simultaneously.
4. Precedence: If one subject must occur before another, their time slots must follow that order.
5. Faculty availability: Exams requiring particular faculty members must respect their availability.
6. Special accommodations: Students requiring accessible facilities or additional arrangements must be assigned suitable examination conditions.

A valid timetable is an assignment satisfying all hard constraints.

ii. Backtracking Search

Backtracking assigns time slots and halls to courses one at a time.

The procedure is:

1. Select an unassigned course.
2. Choose an available time slot and hall.
3. Check whether the assignment satisfies all constraints.
4. If valid, continue with the next course.

5. If no valid assignment is possible, undo the previous assignment and try another option.
6. Stop when a complete valid timetable is found or all possibilities have been explored.

iii. Constraint Propagation and Forward Checking

Forward checking reduces the search space after each assignment.

For example, if Course A is scheduled at 10 AM, the system removes 10 AM from the available domains of all courses that share students with Course A.

Benefits include:

- Detecting conflicts early.
- Reducing unnecessary backtracking.
- Eliminating invalid time-slot choices.
- Improving the efficiency of timetable generation.

Other techniques, such as maintaining arc consistency, can further reduce the available domains.

iv. Real-World Challenges and Scalability

Large universities may have thousands of students, hundreds of courses, limited halls, and complex accommodation requirements. New courses or faculty absences may also require timetable changes.

Recommended strategy: Use CSP with backtracking, the Minimum Remaining Values (MRV) heuristic, degree heuristic, and forward checking.

- MRV selects the course with the fewest remaining valid choices.
- The degree heuristic prioritizes courses that constrain many other courses.
- Forward checking removes invalid future choices early.
- Local search can improve the timetable's quality after a valid solution is found.

This combination can produce valid schedules more efficiently while supporting changes in university requirements.

5. Strategic Game AI Using Minimax and Alpha-Beta Pruning

i. Minimax Algorithm

Minimax is used in competitive games where one player tries to maximize the score while the opponent tries to minimize it.

The algorithm builds a game tree:

- MAX nodes: The AI selects the move with the highest expected score.
- MIN nodes: The opponent selects the move with the lowest expected score.
- Leaf nodes: An evaluation function estimates the value of the resulting game state.

The algorithm assumes that both players choose their moves optimally. It selects the move that provides the best outcome against the opponent's possible responses.

ii. Computational Challenges

A game may have many legal moves at each turn and a large number of possible future states.

If the branching factor is b and the search depth is d, the approximate number of nodes in a full game tree is:

$$ O(b^d) $$

As the search depth increases, the computation required grows exponentially.

This creates several problems:

- Decisions may take too long.
- Memory usage may increase.
- The AI may not examine important future moves within the time limit.
- Complex game states may be difficult to evaluate accurately.

iii. Alpha-Beta Pruning

Alpha-Beta pruning improves Minimax by eliminating branches that cannot affect the final decision.

It maintains two values:

- α: The best value MAX can guarantee so far.
- β: The best value MIN can guarantee so far.

Pruning occurs when:

$$ \alpha \geq \beta $$

Example:

Suppose MAX has already found a move worth 8 points. While examining another move, MIN finds a possible response worth 5 points. If MIN can force that branch to a value of 5 or less, MAX will not select it over the existing move worth 8. The remaining unnecessary branches can therefore be skipped.

With good move ordering, Alpha-Beta pruning can substantially reduce the number of nodes examined. In the best case, its time complexity approaches $O(b^{d/2})$, while the worst case remains $O(b^d)$.

iv. Evaluation Function

A simple evaluation function for a strategy game is:

$$\begin{aligned} E(s)={}&w_1M+w_2U+w_3R\\ &+w_4T+w_5P \end{aligned}$$

Where:

- M = military strength.
- U = number or value of useful units.
- R = available resources.
- T = territory controlled.
- P = progress towards the game objective.
- w_1,w_2,w_3,w_4,w_5 = weights representing the importance of each factor.

The weights should be tuned to the game's rules and objectives. Threats, defensive strength, remaining time, and immediate victory or defeat should also be considered when relevant.

v. Recommended Strategy

Use Iterative Deepening Minimax with Alpha-Beta pruning and a heuristic evaluation function.

The AI searches to increasing depths, retaining the best move from the last completed search. If the time limit is reached, it can return that move instead of waiting for a deeper search to finish.

Additional improvements include:

- Prioritizing promising moves.
- Using a transposition table to reuse previously evaluated states.
- Applying a strict computation budget.
- Giving priority to immediate wins and urgent defensive actions.

This strategy balances decision quality and response time. Alpha-Beta pruning preserves the Minimax result for the searched depth, while iterative deepening and move ordering help the AI make strong decisions under real-time constraints.

Conclusion

These five scenarios demonstrate how different AI search and optimization techniques solve practical problems.

1. Emergency drone delivery: A* with real-time replanning balances route cost and estimated distance.
2. Traffic signal optimization: Genetic Algorithms combined with local optimization can handle large, changing search spaces.
3. Mars rover exploration: Online heuristic-guided search supports decisions in unknown environments.
4. Exam timetable generation: CSP with backtracking and constraint propagation efficiently manages scheduling restrictions.
5. Strategy game AI: Iterative Deepening Minimax with Alpha-Beta pruning improves decision quality while respecting time limits.

The most suitable algorithm depends on the problem's constraints, available information, computational resources, and need for safety, speed, or optimality.